De La Salle University- Manila
Gokongwei College of Engineering

Project Number : 2
Project Title : Ping Pong Game

Date Performed : 4/14/2026
Date Submitted : 4/14/2026

Instructor : Dr. Argel A. Bandala

Name of Student :

- Galvez, Ron John B.
- Metrillo, Brian David
- Romero, John Mark
- Villanueva, Jaz V.

Student ID :

- 12334731
- 12300160
- 12313726
- 12314536

Subject / Section : LBYCPM3

Objectives

- Design and implement a ping pong game using PIC16F877A microcontroller
- Develop a schematic diagram of the ping pong system
- Create and compile a working MikroC code for the PIC microcontroller
- Simulate the circuit using Proteus 8.11
- Develop a PCB design in KiCad based on schematic diagram

Introductory Information

This project focuses on designing and building physical hardware for a ping pong game using a PIC16F877A 8-bit microcontroller. PIC16F877A was utilized for this project for the microcontroller supports enough I/O pins and memory for implementing basic systems such as this pong game system [1]. To create and compile the working code for the PIC microcontroller, the MikroC environment is utilized [2].

The game will utilize an 8x8 matrix LED display for outputting the ball and platform which is interfaced through a MAX7219 chip. The MAX7219 handles the multiplexing for the LEDs [3], and is interfaced to the PIC microcontroller using Serial Peripheral Interface (SPI) protocol that involves five connections namely VCC, GND, DIN, CS (LOAD), and CLK. These pin connections play a crucial role in making the PIC communicate with the matrix display.

For the inputs of the game, 6mm buttons are utilized so the players can control the movement of their platforms. The system accounts for debouncing to make sure the microcontroller does not register unintended multiple presses from the physical buttons.

Furthermore, the project involves simulating the circuit using Proteus 8.11 to check if the schematic diagram and code are working [4]. After a successful simulation, a PCB design is developed in KiCad based on the schematic diagram to complete the hardware implementation [5].

Materials Needed:

- 1x 8x8 Matrix Display
- 1x MAX7219 Driver
- 5x 10 kiloohm resistor
- 1x PIC16F877A
- 1x PICKit2 with USB Cable
- Computer with PICKit2 programmer and MikroC
- 5x 6mm buttons
- Solid core wires/jumper wires
- 5V DC Power Supply

- 1x 16MHz Crystal Oscillator
- 2x 22pF capacitor

Procedure

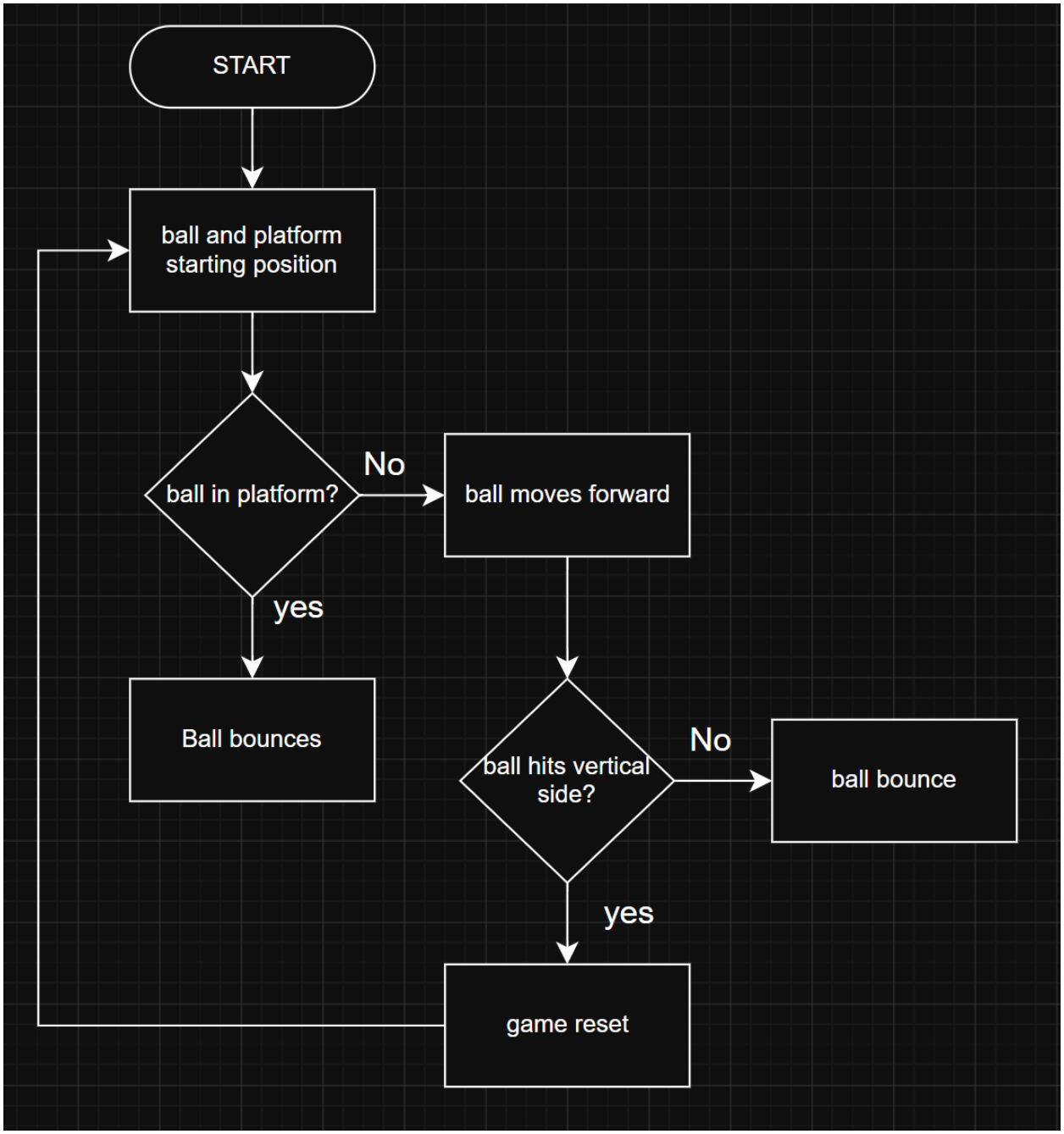
1. Circuit Design and Simulation in Proteus
    a. Open Proteus and create a new schematic project.
    b. Add the PIC16F877A from the component library and attach an 8 MHz crystal oscillator to the OSC1 and OSC2 pins, with two 22 pF capacitors to ground.
    c. Add a MAX7219 LED driver and an 8x8 LED matrix display. Connect the DIN pin of the MAX7219 to RC5 (SDO), the CLK pin to RC3 (SCK), and the CS pin to RC0 of the PIC.
    d. Connect the four player buttons to RB0, RB1, RB2, and RB3. Each button should have a 10 kilohm pull-down resistor to ground so that the pin reads low when the button is not pressed and high when it is pressed.
    e. Connect a reset button (MCLR) to the MCLR/Vpp pin with a 10 kilohm pull-up resistor to VDD.
    f. Add a 5V power supply to VDD and connect all VSS pins to ground.
    g. Verify all schematic connections and save the Proteus project.
2. Firmware Development in MikroC
    a. Open MikroC Pro for PIC and create a new project. Set the device to PIC16F877A and the oscillator to 8 MHz.
    b. Write the MAX7219 initialization function (MAX7219_Init) to configure RC0 as the CS pin output and set up hardware SPI using SPI1_Init_Advanced with master mode, 16 clock divisor, idle low, and low-to-high clock edge.
    c. Write MAX7219_Config to send the startup register values: disable decode mode, set intensity to mid-level (0x08), enable all 8 scan digits, turn off display test, and set normal operation mode.
    d. Write MAX7219_Send to handle a single SPI register write: pull CS low, send the register address and data bytes, then pull CS high.
    e. Define the five paddle states as bitmasks (STATE_1 through STATE_5), each representing a 4-row window of the 8-row matrix that shifts up or down in response to button presses.
    f. Write ButtonGoUp and ButtonGoDown to handle paddle movement using a switch-case on the current paddle state, clamping at the top and bottom limits.
    g. Write UpdateBallPosition to check for paddle collisions by ANDing the ball's current row bitmask against the paddle state at the matching column. If there is a hit, reverse ballDirX. Then advance the ball one step in both X and Y directions, bouncing the Y direction when the ball reaches row 0 or row 7. Set isScore to true if the ball exits the board at column 1 or column 8.
    h. Write RenderFrame to clear all columns, then draw the paddles and the ball. If the ball is on the same column as a paddle, OR its bitmask into the paddle's row value before sending to the MAX7219.

i. Write InitGame to reset both paddles to STATE_3, reset ball position and direction, clear isScore, and call RenderFrame.
j. Write the main loop. After initialization, call InitGame then enter the inner game loop. Each iteration calls UpdateBallPosition, then runs a 1000 ms inner timing loop that reads and debounces all four buttons using edge detection against previous button states. Each inner iteration delays 8 ms and increments the elapsed counter. When isScore becomes true, exit the inner loop, wait 250 ms, and restart the game with InitGame.
k. Compile the firmware in MikroC. Fix any errors and recompile until a valid .hex file is produced.

3. Simulation and Testing
   a. Load the .hex file into the PIC component in Proteus by double-clicking the component and pointing to the compiled .hex file path.
   b. Run the simulation and confirm that both paddles appear centered on their respective columns (column 7 for Player 1, column 2 for Player 2) and that the ball starts moving.
   c. Test paddle movement by pressing the buttons for Player 1 and Player 2. Confirm that each paddle shifts up or down on the matrix and stops at the edges.
   d. Let the ball travel toward a paddle and confirm it bounces back when it hits. Let it past the paddle and confirm the game resets after a brief pause.
   e. Take screenshots of the simulation for documentation.
4. PCB Schematic in KiCad
   a. Open KiCad and create a new project. Open the schematic editor and place all components: the PIC16F877A, the 5-pin SPI connector (J3) for the MAX7219 module, the 2-pin power connector (J2), the four player buttons (SW1 to SW4) with their 10 kilohm pull-down resistors (R2 to R5), the MCLR reset button (SW5) with pull-up resistor (R1), and the 8 MHz crystal (Y1) with its two 22 pF load capacitors (C3 and C5).
   b. Wire all nets following the same topology as the Proteus schematic. Connect the SPI lines (RC3, RC4, RC5) and CS (RC0) to the 5-pin connector. Connect the button pins to RB0 through RB3.
   c. Run the electrical rules check (ERC) and fix any flagged issues before proceeding.
   d. Open the PCB editor and import the netlist. Arrange components in a compact layout. Route all traces, keeping the SPI lines short and grouping the button resistors near their respective switches.
   e. Run the design rules check (DRC) to confirm no clearance violations.
   f. Export the schematic and PCB screenshots for documentation.
5. Physical Implementation
   a. Assemble the circuit on a breadboard following the KiCad schematic.
   b. Connect the PIC16F877A, crystal, capacitors, and MCLR button first. Then wire the buttons and pull-down resistors to PORTB.

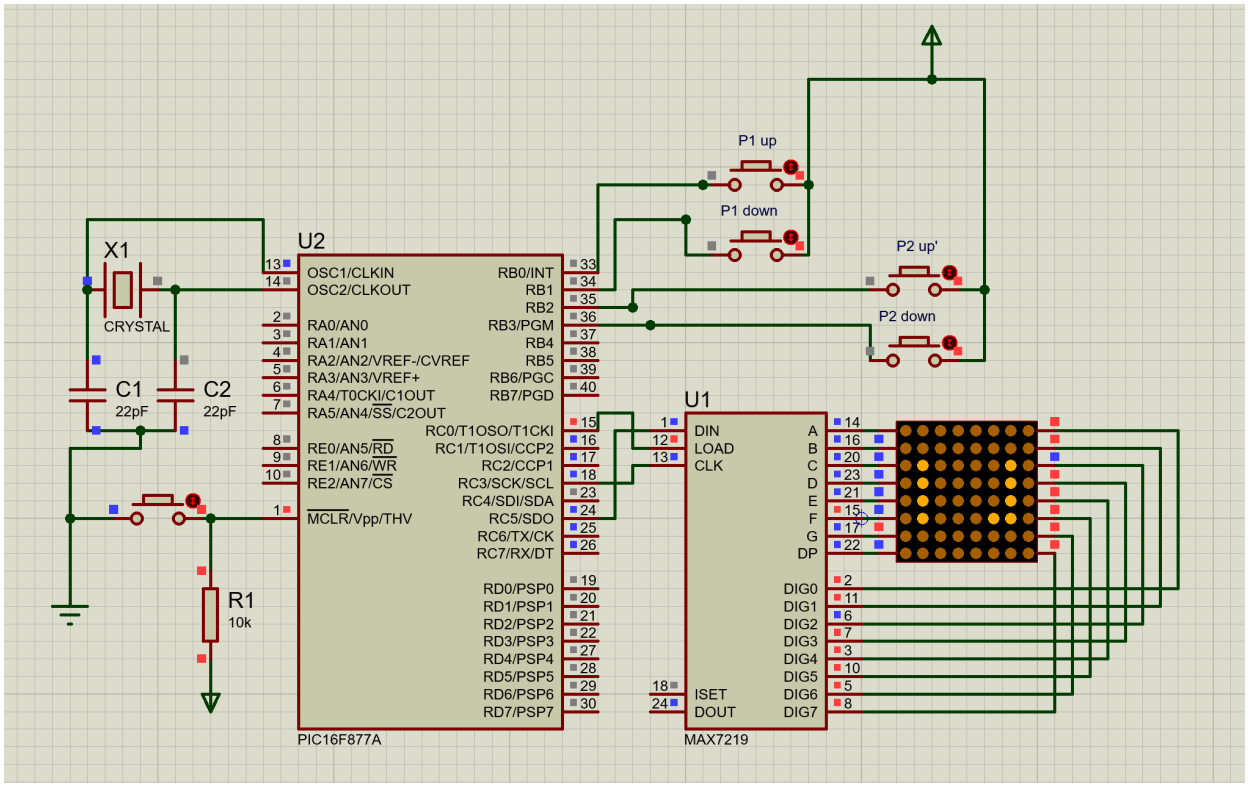
c. Connect the SPI lines from PORTC to the MAX7219 module and then connect the 8x8 LED matrix to the MAX7219 outputs.
d. Supply 5V DC power to the circuit.
e. Flash the compiled .hex file to the PIC using PICKit2 and the MikroC programmer tool.
f. Power on the circuit and verify that the game runs correctly. Test both paddles and confirm the ball bounces and the game resets on a score.
g. Once breadboard version works, transfer everything to a universal PCB via soldering. Refer to traces made in KiCad PCB layout. Repeat steps B to F on the universal PCB.
h. Take photos of the working physical implementation for documentation.

Firmware Flowchart

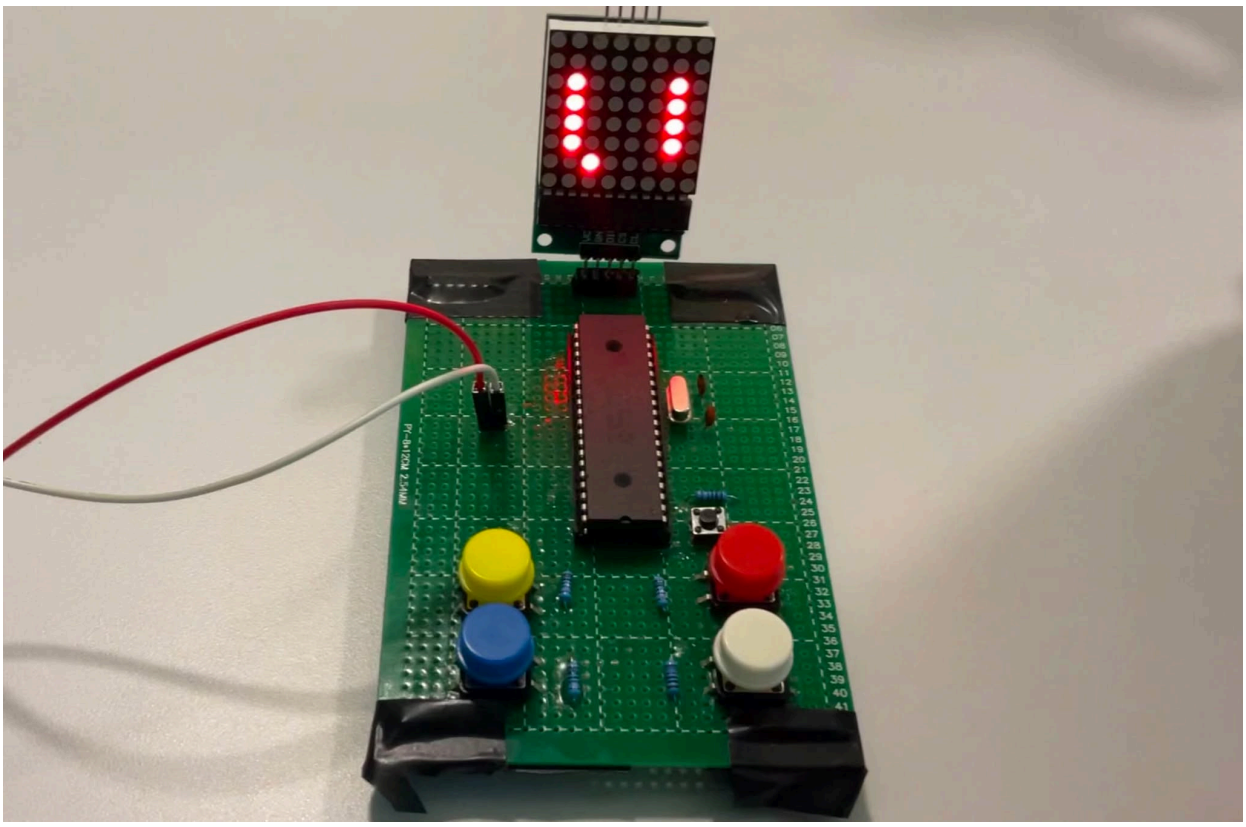


Data and Results

Proteus Schematic

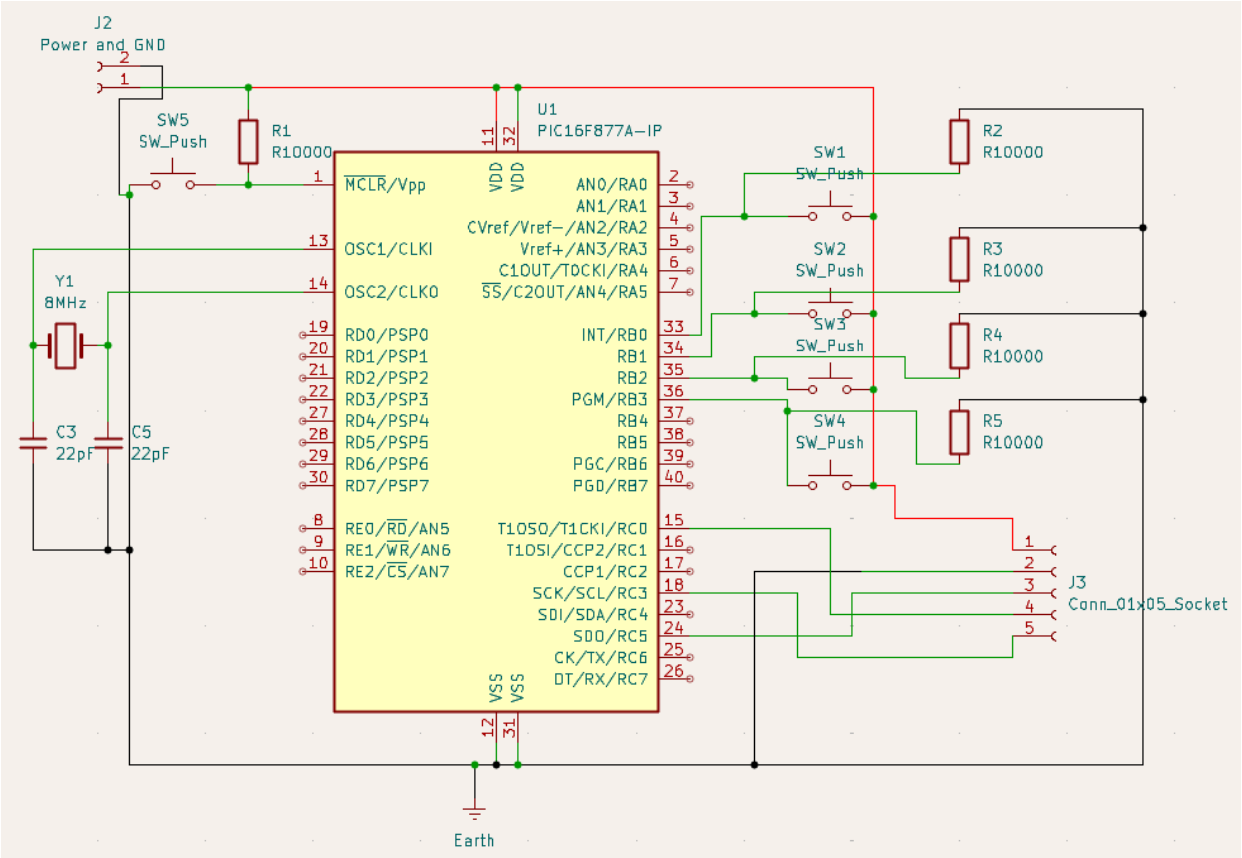


Physical Implementation

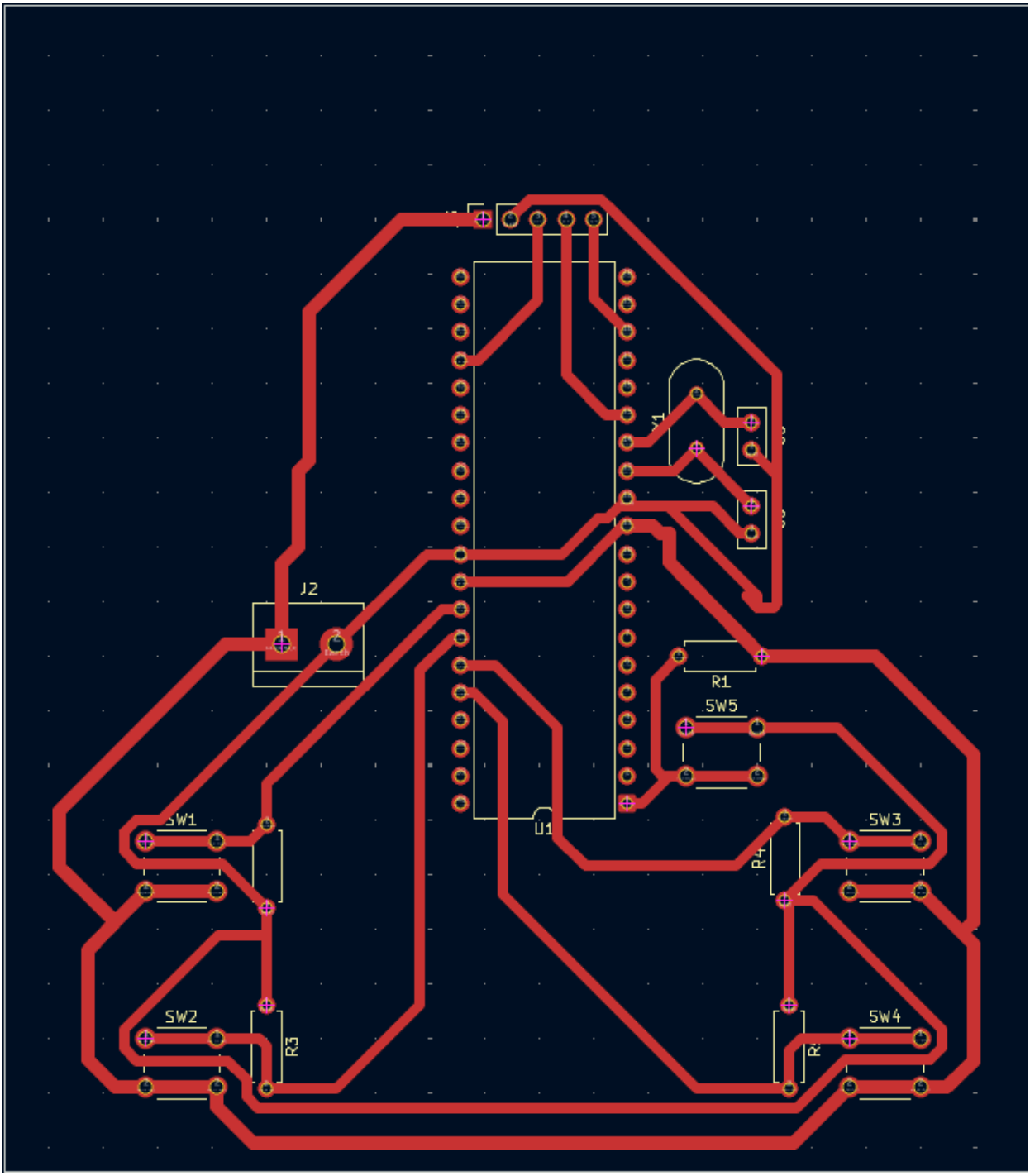


Video Demonstration:
https://drive.google.com/file/d/1vLUG_qZoW9VsrUOYHv4MvmWK9TPfnm_L/view?usp=drivesdk

KiCad Schematic



KiCad PCB Design



MikroC Code

/*

```
* File:   PONG_PROJ.c
* Pong Game - PIC16F877A + MAX7219 8x8 LED Matrix
* MikroC Pro for PIC
*
* Wiring (Hardware SPI):
*   RC5/SDO -> DIN
*   RC3/SCK -> CLK
*   RC0     -> CS
*   RB0     -> Player 1 Up    (Active High)
*   RB1     -> Player 1 Down  (Active High)
*   RB2     -> Player 2 Up    (Active High)
*   RB3     -> Player 2 Down  (Active High)
*/

// ===================== Bool
================================================
#define bool  unsigned char
#define true  1
#define false 0

// ===================== Button Pins ========================================
#define Player1_up_pin   RB3_bit
#define Player1_down_pin RB2_bit
#define Player2_up_pin   RB1_bit
#define Player2_down_pin RB0_bit
```

```
#define BUTTON_PRESSED(pin) ((pin) == 1)

// ===================== MAX7219 Register Addresses =========================
#define NO_OP_REG           0x00
#define DECODE_MODE_REG     0x09
#define INTENSITY_REG       0x0A
#define SCAN_LIMIT_REG      0x0B
#define SHUTDOWN_REG        0x0C
#define DISPLAY_TEST_REG    0x0F

// ===================== MAX7219 Configuration Values =======================
#define DISABLE_DECODE      0x00
#define BRIGHTNESS          0x08
#define SCAN_ALL_DIGITS     0x07
#define NORMAL_OPERATION    0x01
#define DISABLE_TEST        0x00

// ===================== MAX7219 CS Pin
=====================================
#define MAX7219_CS      RC0_bit
#define MAX7219_CS_DIR  TRISC0_bit

// ===================== Board / Timing =====================================
#define PLAYER1_COL         7
```

```
#define PLAYER2_COL         2

#define FRAME_DELAY_MS      1000

#define DEBOUNCE_DELAY_MS   8


// ===================== LED Row Bitmasks
===================================

#define No_Light  0x00

#define Row0      0x01

#define Row1      0x02

#define Row2      0x04

#define Row3      0x08

#define Row4      0x10

#define Row5      0x20

#define Row6      0x40

#define Row7      0x80


// ===================== Paddle States (4-row paddle) =======================

#define STATE_5   0xF0

#define STATE_4   0x78

#define STATE_3   0x3C

#define STATE_2   0x1E

#define STATE_1   0x0F


// ===================== Global Game Variables ==============================

unsigned char p1State = STATE_3;
```

```
unsigned char p2State = STATE_3;


int ballDirX = -1;
int ballDirY = 1;
int ballX = 4;
unsigned char ballYState = Row4;


bool isScore = false;


unsigned char prevB0 = 0;
unsigned char prevB1 = 0;
unsigned char prevB2 = 0;
unsigned char prevB3 = 0;


// ====================== MAX7219 Low-Level
==================================
void MAX7219_Send(unsigned char regAddr, unsigned char regData) {
    MAX7219_CS = 0;
    SPI1_Write(regAddr);
    SPI1_Write(regData);
    MAX7219_CS = 1;
}


void MAX7219_Clear() {
    unsigned char col;
```

```
    for (col = 1; col <= 8; col++) {
        MAX7219_Send(col, No_Light);
    }
}

void MAX7219_Config() {
    MAX7219_Send(DISPLAY_TEST_REG, DISABLE_TEST);
    MAX7219_Send(SHUTDOWN_REG, NORMAL_OPERATION);
    MAX7219_Send(DECODE_MODE_REG, DISABLE_DECODE);
    MAX7219_Send(INTENSITY_REG, BRIGHTNESS);
    MAX7219_Send(SCAN_LIMIT_REG, SCAN_ALL_DIGITS);
}

void MAX7219_Init() {
    MAX7219_CS_DIR = 0;
    MAX7219_CS = 1;

    TRISC3_bit = 0;   // SCK output
    TRISC5_bit = 0;   // SDO output

    RC3_bit = 0;
    RC5_bit = 0;

    SPI1_Init_Advanced(
        _SPI_MASTER_OSC_DIV16,
```

```
        _SPI_DATA_SAMPLE_MIDDLE,
        _SPI_CLK_IDLE_LOW,
        _SPI_LOW_2_HIGH
    );
}

// ====================== Game Display
========================================
void ApplyPaddleState(unsigned char playerCol, unsigned char paddleState) {
    MAX7219_Send(playerCol, paddleState);
}

void RenderFrame() {
    unsigned char col;
    unsigned char p1Render;
    unsigned char p2Render;

    for (col = 1; col <= 8; col++) {
        MAX7219_Send(col, No_Light);
    }

    p1Render = p1State;
    p2Render = p2State;

    if (ballX == PLAYER1_COL) p1Render |= ballYState;
```

```
    if (ballX == PLAYER2_COL) p2Render |= ballYState;

    ApplyPaddleState(PLAYER1_COL, p1Render);
    ApplyPaddleState(PLAYER2_COL, p2Render);

    if ((ballX != PLAYER1_COL) && (ballX != PLAYER2_COL)) {
        MAX7219_Send((unsigned char)ballX, ballYState);
    }
}

void InitGame() {
    p1State = STATE_3;
    p2State = STATE_3;
    ballX = 5;
    ballYState = Row4;
    ballDirX = -1;
    ballDirY = 1;
    isScore = false;

    MAX7219_Clear();
    RenderFrame();
}

// ===================== Button Reading ====================================
unsigned char ReadButton0() { return BUTTON_PRESSED(Player1_up_pin); }
```

```
unsigned char ReadButton1() { return BUTTON_PRESSED(Player1_down_pin); }
unsigned char ReadButton2() { return BUTTON_PRESSED(Player2_up_pin); }
unsigned char ReadButton3() { return BUTTON_PRESSED(Player2_down_pin); }

// ===================== Paddle Movement
====================================
void ButtonGoUp(unsigned char player) {
    if (player == 1) {
        switch (p1State) {
            case STATE_1: p1State = STATE_2; break;
            case STATE_2: p1State = STATE_3; break;
            case STATE_3: p1State = STATE_4; break;
            case STATE_4: p1State = STATE_5; break;
            case STATE_5: p1State = STATE_5; break;
            default:      p1State = STATE_3; break;
        }
    } else {
        switch (p2State) {
            case STATE_1: p2State = STATE_2; break;
            case STATE_2: p2State = STATE_3; break;
            case STATE_3: p2State = STATE_4; break;
            case STATE_4: p2State = STATE_5; break;
            case STATE_5: p2State = STATE_5; break;
            default:      p2State = STATE_3; break;
        }
```

```
    }
}

void ButtonGoDown(unsigned char player) {
    if (player == 1) {
        switch (p1State) {
            case STATE_1: p1State = STATE_1; break;
            case STATE_2: p1State = STATE_1; break;
            case STATE_3: p1State = STATE_2; break;
            case STATE_4: p1State = STATE_3; break;
            case STATE_5: p1State = STATE_4; break;
            default:      p1State = STATE_3; break;
        }
    } else {
        switch (p2State) {
            case STATE_1: p2State = STATE_1; break;
            case STATE_2: p2State = STATE_1; break;
            case STATE_3: p2State = STATE_2; break;
            case STATE_4: p2State = STATE_3; break;
            case STATE_5: p2State = STATE_4; break;
            default:      p2State = STATE_3; break;
        }
    }
}
```

```
// ===================== Ball Logic ========================================
void UpdateBallPosition() {
    if (ballX == PLAYER2_COL) {
        if (p2State & ballYState) {
            ballDirX = -ballDirX;
        }
    } else if (ballX == PLAYER1_COL) {
        if (p1State & ballYState) {
            ballDirX = -ballDirX;
        }
    }

    ballX += ballDirX;

    if ((ballYState & Row0) == Row0) {
        ballYState = Row1;
        ballDirY = 1;
    } else if ((ballYState & Row1) == Row1) {
        ballYState = (ballDirY == 1) ? Row2 : Row0;
    } else if ((ballYState & Row2) == Row2) {
        ballYState = (ballDirY == 1) ? Row3 : Row1;
    } else if ((ballYState & Row3) == Row3) {
        ballYState = (ballDirY == 1) ? Row4 : Row2;
    } else if ((ballYState & Row4) == Row4) {
        ballYState = (ballDirY == 1) ? Row5 : Row3;
```

```
    } else if ((ballYState & Row5) == Row5) {
        ballYState = (ballDirY == 1) ? Row6 : Row4;
    } else if ((ballYState & Row6) == Row6) {
        ballYState = (ballDirY == 1) ? Row7 : Row5;
    } else if ((ballYState & Row7) == Row7) {
        ballYState = Row6;
        ballDirY = -1;
    }

    if ((ballX == 1) || (ballX == 8)) {
        isScore = true;
    }

    RenderFrame();
}

// ===================== Main
================================================
void main() {
    unsigned int elapsed;
    unsigned char b0, b1, b2, b3;

    TRISB = 0xFF;
    TRISC = 0x00;
    PORTB = 0x00;
```

```
PORTC = 0x00;

OPTION_REG |= 0x80;   // disable PORTB pull-ups

MAX7219_Init();
MAX7219_Config();

while (1) {
    InitGame();

    while (!isScore) {
        elapsed = 0;
        UpdateBallPosition();

        while (elapsed < FRAME_DELAY_MS) {
            b0 = ReadButton0();
            b1 = ReadButton1();
            b2 = ReadButton2();
            b3 = ReadButton3();

            if (b0 && !prevB0) {
                ButtonGoUp(1);
                RenderFrame();
            }
            if (b1 && !prevB1) {
```

```
            ButtonGoDown(1);
            RenderFrame();
        }
        if (b2 && !prevB2) {
            ButtonGoUp(2);
            RenderFrame();
        }
        if (b3 && !prevB3) {
            ButtonGoDown(2);
            RenderFrame();
        }

        prevB0 = b0;
        prevB1 = b1;
        prevB2 = b2;
        prevB3 = b3;

        Delay_ms(DEBOUNCE_DELAY_MS);
        elapsed += DEBOUNCE_DELAY_MS;
      }
    }

    Delay_ms(250);
  }
}
```

Discussion

Our project is a two-player pong game built on the PIC16F877A microcontroller, using a MAX7219 LED driver to control an 8x8 LED matrix as the display. Player 1 controls the right paddle at column 7 using RB3 (up) and RB2 (down), while Player 2 controls the left paddle at column 2 using RB1 (up) and RB0 (down). The ball moves diagonally across the matrix, bouncing off the top and bottom edges and reversing direction when it hits a paddle. If the ball passes a paddle and exits the board, the game resets.

To drive the 8x8 matrix, we used the MAX7219, a serial LED driver that handles all the row scanning internally. It communicates with the PIC over hardware SPI on the PORTC pins, which keeps the firmware simple since we only need to write a column register value to update that column on the display. The CS pin (RC0) is manually toggled before and after each SPI transfer:

```C/C++
void MAX7219_Send(unsigned char regAddr, unsigned char regData) {
    MAX7219_CS = 0;
    SPI1_Write(regAddr);
    SPI1_Write(regData);
    MAX7219_CS = 1;
}
```

We configured the MAX7219 on startup to disable BCD decode mode (since we are directly writing bitmasks), set a medium brightness, and enable all 8 rows via the scan limit register. This only needs to be done once in MAX7219_Config during initialization.

Each paddle is stored as a single byte bitmask representing which of the 8 rows in its column are lit. Since the paddle covers 4 rows at a time, there are exactly 5 valid positions it can be in:

```C/C++
STATE_1 = 0x0F  (rows 0-3, bottom of matrix)
STATE_2 = 0x1E  (rows 1-4)
STATE_3 = 0x3C  (rows 2-5, center)
STATE_4 = 0x78  (rows 3-6)
STATE_5 = 0xF0  (rows 4-7, top of matrix)
```

Moving the paddle up or down just transitions between these states using a switch-case. The paddle is clamped at STATE_1 and STATE_5 so it cannot go off screen. Both paddles start at STATE_3 at the center.

This bitmask approach is what makes collision detection straightforward. To check if the ball hit a paddle, we just AND the ball's row bitmask against the paddle's current state:

```
C/C++
if (p1State & ballYState) {
    ballDirX = -ballDirX;
}
```

If the result is nonzero, the ball is on a row that the paddle occupies, so we reverse the horizontal direction.

The ball's horizontal position is tracked as an integer column (ballX), and its vertical position is tracked as a row bitmask (ballYState) using the same Row0 through Row7 definitions used for the paddles. Each call to UpdateBallPosition checks for paddle collisions, then moves the ball by adding ballDirX to ballX and shifting ballYState in the direction of ballDirY. When the ball reaches Row0 or Row7, the vertical direction is reversed:

```
C/C++
} else if ((ballYState & Row7) == Row7) {
    ballYState = Row6;
    ballDirY = -1;
}
```

If ballX goes to 1 or 8, the ball has left the playfield and isScore is set to true, ending the round.

Instead of using timer interrupts, the game uses a simple polling loop with Delay_ms for timing. Each frame, UpdateBallPosition is called once to move the ball, then a 1000 ms inner loop runs in 8 ms steps. During each step, all four buttons are read and checked for rising edges using edge detection against the previous button state:

```
C/C++
if (b0 && !prevB0) {
    ButtonGoUp(1);
    RenderFrame();
}
```

The 8 ms delay between reads also acts as a basic software debounce, which is the same approach used in Project 1. The edge detection (comparing current state to the previous state) ensures that a held button only registers one press per frame cycle.

This approach is simpler than interrupt-driven input but has one tradeoff: the ball speed is fixed at roughly one move per second and cannot be changed without adjusting FRAME_DELAY_MS. The button polling is responsive enough within each frame because the inner loop runs 125 iterations in one second (1000 ms / 8 ms), so button presses are detected quickly.

The RenderFrame function clears all 8 columns first by writing 0x00 to each, then redraws the paddles and the ball. One edge case that needed handling is when the ball is on the same column as a paddle. In that case, the ball's row bitmask is OR'd into the paddle's state before sending to the MAX7219, so both the paddle and the ball are visible on that column at the same time:

```
C/C++
if (ballX == PLAYER1_COL) p1Render |= ballYState;
```

This prevents the ball from being invisible when it is directly on a paddle column, which would make the collision harder to see visually.

Overall, the game worked correctly in simulation and on the actual hardware. One thing we noticed is that the fixed 1-second frame delay makes the game feel quite slow. A shorter delay, maybe 200 to 300 ms, would make it more playable. The game also does not distinguish which player scored, it just resets. Adding a score counter displayed before each round, or using the LED matrix to flash a short pattern, would make the game more complete. The PCB layout was also a bit large due to the button placement around the board edges, but it kept the routing clean without any overlapping traces.

Analysis and conclusion

The ping pong game system was successfully implemented using the PIC16F877A microcontroller, MAX7219 LED driver, and an 8x8 LED matrix display. Based on the simulation in Proteus, the prototyped PCB in KiCad, and the actual hardware output shown in the results section, the system behaves as expected. The paddles respond correctly to the button inputs, the ball moves consistently across the matrix, and collisions with paddles and boundaries are properly detected.

One of the strengths of the system is using MAX7219 for driving the LED matrix. Since the driver handles multiplexing internally, the microcontroller only needs to send simple SPI commands, which makes the firmware easier to manage. The use of bitmasks for representing both the paddles and the ball also made the implementation more efficient. This helped us simplify rendering and allowed collision detection to be done using basic logical operations, which is good for a microcontroller with limited resources.

The button handling also worked reliably due to the combination of edge detection and a small delay for debouncing. This ensured that each press is only registered once per cycle, preventing unintended multiple movements. As a result, the controls feel consistent during gameplay.

However, some limitations were observed. The most noticeable issue is the slow game speed caused by the fixed 1000 ms frame delay. While this makes the timing easier to implement using a polling loop, it also makes the game less engaging. Reducing this delay or using a timer-based approach could improve responsiveness. Another limitation is that the system does not track or display scores. The game simply resets when the ball exits the playfield, which makes it feel incomplete. Additionally, the PCB layout is functional but relatively large, suggesting that component placement can still be optimized.

Overall, the project met all the stated objectives, including designing the circuit, developing the firmware, simulating the system, and implementing it on actual hardware. The system demonstrates proper integration of hardware and software components and shows a clear understanding of microcontroller-based design. While there are areas for improvement, the current implementation is stable and functional, and it provides a solid foundation for further enhancements.

References

[1] Microchip Technology, "PIC16F87XA Data Sheet," Microchip Technology Inc., 2003. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/39582b.pdf

[2] MikroElektronika, "MikroC PRO for PIC - User Manual," MikroElektronika, 2011. [Online]. Available:
https://download.mikroe.com/documents/compilers/mikroc/pic/mikroc-pic-manual-v701.pdf

[3] Analog Devices, "MAX7219/MAX7221 Serially Interfaced, 8-Digit LED Display Drivers," Rev. 4, Analog Devices Inc., 2003. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/MAX7219-MAX7221.pdf

[4] Labcenter Electronics, "Proteus Design Suite User Manual," Labcenter Electronics Ltd. [Online]. Available: https://www.labcenter.com/

[5] KiCad Developers Team, "KiCad Documentation," KiCad EDA. [Online]. Available: https://docs.kicad.org/